

Playing with the Rubik's Cube and its God's number in Rocq

Laurent Théry

INRIA, Stamp Team
Laurent.Theiry@inria.fr

Abstract. One needs no more than 20 turns to solve any Rubik's cube position. This is called God's number and was computed in 2010. This note presents three smaller results about the Rubik's cube that are proved in Rocq. First, we prove that twenty is a lower bound. Second, we change metric (counting half turns as two moves) and prove that in this case twenty-six is a lower bound. Finally, the published computation of God's number (35 CPU years) splits the cube into the $2\,217\,093\,120$ cosets of a subgroup, one search to a coset. We formalise the correctness of the coset search, and certify the computation on one such coset.

Keywords. Rubik's cube, God's number, formal proof, Rocq, group theory.

1 The problem

A Rubik's cube is built from 26 small cubes: 8 for the corners with 3 stickers each, 12 for the edges with 2 stickers, and 6 for the centres. The centres are fixed. A turn moves four corners and four edges and rotates a centre.



Figure 1: Anatomy of the cube

Not every arrangement of the pieces can be reached by turning faces. The number of possible arrangements is

$$8! \cdot 3^7 \cdot 12! \cdot 2^{11} / 2 = 43\,252\,003\,274\,489\,856\,000 \approx 4.3 \cdot 10^{19},$$

The eight corners can be in any order ($8!$) and their orientations are free except for the last one (3^7). The twelve edges can be in any order ($12!$) and again their orientations are free except the last one (2^{11}). The result is then halved, because corners and edges are not independent.

Turning one face is a *move*, and a half turn counts as one move just like a quarter turn. How many moves the worst position needs is called *God's number*. Counting a half turn as one move is a choice. A half turn can also count as two moves, and that gives a second number for the same cube. We prove a lower bound for each.

We take one position: the **superflip**. Every cubie is in its own place, but all the edges have the wrong orientation.

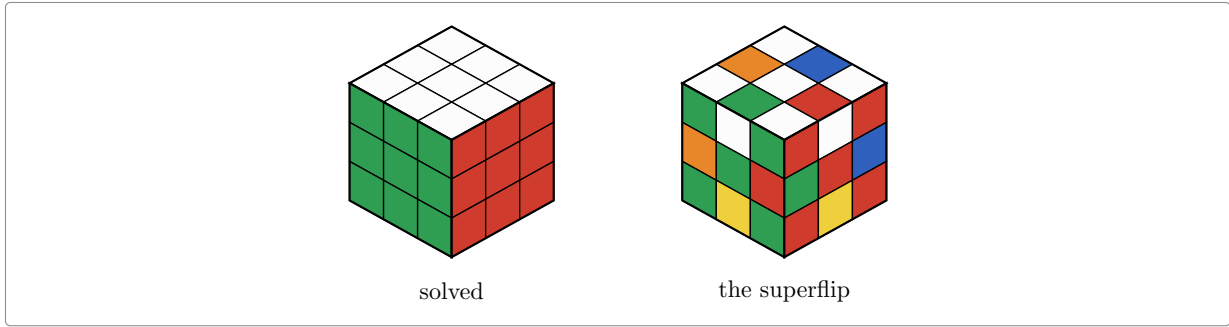


Figure 2: A solved cube, and the superflip.

The superflip is left unchanged by all 48 symmetries of the cube. We use it to prove the lower bound.

2 The cube as permutations

There are several ways of representing the Rubik's cube. We use the one based on stickers. There are six faces of nine stickers, from which we can remove the six centre stickers that never move. We number the stickers from 0 to 47, as in Figure 3 and Figure 4. So a move is a rearrangement of the 48 stickers. Each face has eight of them, taken left to right and top to bottom, with the centre skipped. Up gets 0–7, left 8–15, front 16–23, right 24–31, back 32–39 and down 40–47.

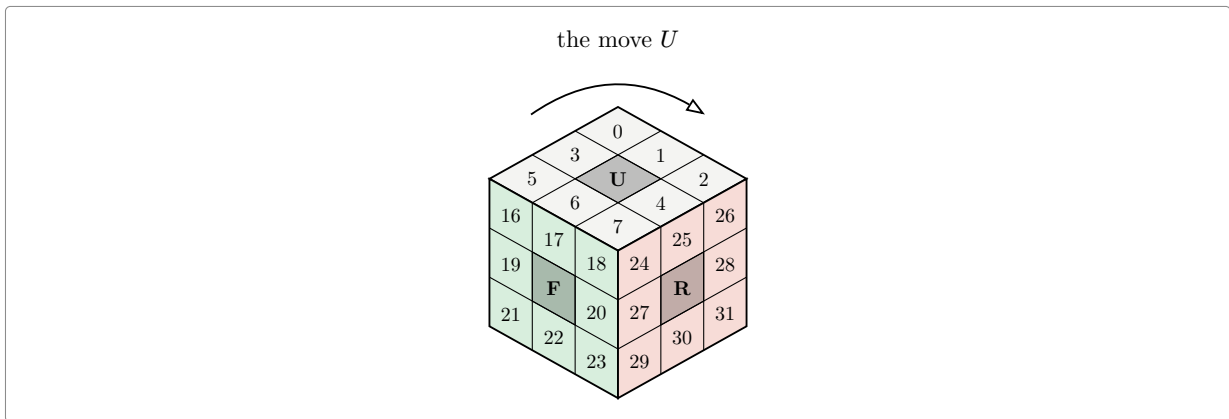


Figure 3: Three of the six faces, and the turn of the top one.

			0	1	2			
			3	U	4			
			5	6	7			
8	9	10	16	17	18	24	25	26
11	L	12	19	F	20	27	R	28
13	14	15	21	22	23	29	30	31
			40	41	42			
			43	D	44			
			45	46	47			

Figure 4: The cube unfolded, with all forty-eight places numbered.

A move is written down by saying where the stickers go. For example, if we turn the top face clockwise, the sticker in corner 0 goes to corner 2, the one in 2 to 7, the one in 7 to 5, and the one in 5 back to

0. This is represented by a four step cycle, written (0 2 7 5). The four edge stickers of that face do the same, (1 4 6 3). The turn does not only move the top face. It also carries the top row of each side face round to the next one: front to left, left to back, back to right, right to front. That is three more cycles, (8 32 24 16) and its two companions. Figure 5 shows all of it, each square saying which sticker sits there afterwards. The top row of the left face holds 16, 17, 18, the stickers that came round from the front.

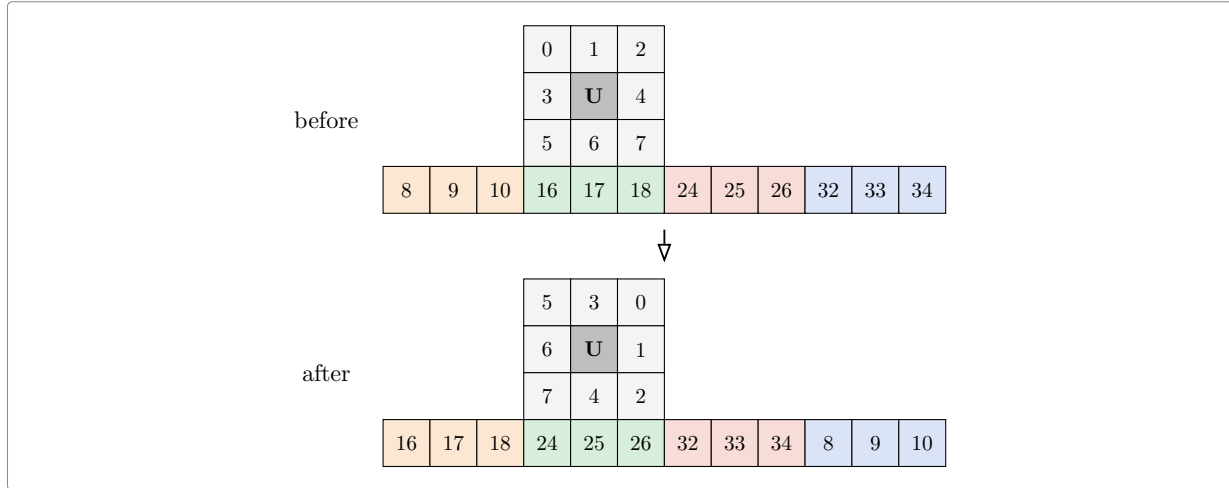


Figure 5: Everything a clockwise turn of the top face moves.

There are six clockwise quarter turns: up, right, front, down, left and back. Each of them can also be done twice or backwards, which gives the *eighteen moves*

U	R	F	D	L	B
U2	R2	F2	D2	L2	B2
U'	R'	F'	D'	L'	B'

A move turned twice is written U2, and a move turned backwards U'. A position is a product of moves. For instance R U R' U', a word of length 4. The set of all positions is a group G : the *cube group*. Solving a position in d moves means writing it as a word of d moves. So “solvable in at most d moves” says that the position lies in the *ball of radius d* around the solved cube. God’s number is the diameter of the Cayley graph.

2.1 The cube in Rocq

We use the Rocq prover [1] and its Mathcomp library [2] for our formalisation. The file, `Rubik333.v`, is a direct transcription of the paragraphs above:

```
Definition facelet := 'I_48.
```

```
Definition Umove : {perm facelet} := cyc [:: 0@; 2@; 7@; 5@] * cyc [:: 1@; 4@;
6@; 3@] * cyc [:: 8@; 32@; 24@; 16@] * cyc [:: 9@; 33@; 25@; 17@] * cyc [:: 10@;
34@; 26@; 18@]. (* ... and five more, one per face ... *)
```

```
Definition faces : seq {perm facelet} :=
  [:: Umove; Rmove; Fmove; Dmove; Lmove; Bmove].
```

```
Definition moves : seq {perm facelet} :=
  flatten [seq [:: g; g ^+ 2; g ^-1] | g <- faces].
```

```
Definition G : {group {perm facelet}} := <<Sset>>.
```

Line by line:

- 'I_48 is the type of numbers **below** 48.
- {perm facelet} is the type of *permutations* of those facelets.

- `cyc [:: 0@; 2@; 7@; 5@]` is the *cycle* that sends 0 to 2, 2 to 7, 7 to 5 and 5 back to 0, leaving the other forty-four places where they are. The @ is local notation turning a plain number into a facelet.
- The infix symbol `*` composes two permutations, so `Umove` is the cycles of Figure 5 put together, and `g ^+ 2` and `g ^-1` are the turn done twice and undone. Note that the composition order is the opposite of the usual one: Mathcomp applies permutations on the right, so `(g * m) f` is `m (g f)`.
- `seq` is a list, and `faces` is the list of the six clockwise quarter turns. `moves` runs through it and keeps three moves per face, which is the eighteen.
- `<<Sset>>` is the group generated by a set: everything reachable by composing moves. This is the cube group.

The superflip is defined in `Diameter.v` as the twelve swaps that exchange the two stickers of each edge:

```
Definition Spcyc : seq (seq facelet) :=
  [:: [:: 1@; 33@]; [:: 3@; 9@]; [:: 4@; 25@];
    (* ... nine more, one per edge ... *) ].
```

```
Definition superflip : {perm facelet} := \prod_(l <- Spcyc) cyc l.
```

That makes it a permutation but nothing says a legal position. For this, we have to prove that it belongs to G . The proof is one equality. Here is the word of twenty moves we use to witness it:

U R2 F B R B2 R U2 L B2 R U' D' R2 F R' L B2 U2 F2

3 Searching for the lower bound

We want to show that the superflip cannot be solved in nineteen moves or less. The naive approach is to try every word of at most 19 moves : starting from the superflip: if the solved cube never turns up, a lower bound is 20. A tree of depth 19 with a branching of 18 is 18^{19} words, far too many to make this naive search practical. We need to refine it.

3.1 The pruning estimate

A first refinement is to figure out a test that cuts in advance the branches which cannot succeed. For that, we associate at each position a coarse lower bound h on the number of moves still needed. If h is 20 and only 18 moves remain, the branch cannot reach the solved cube in time. It is cut, with everything below it, as Figure 6 shows.

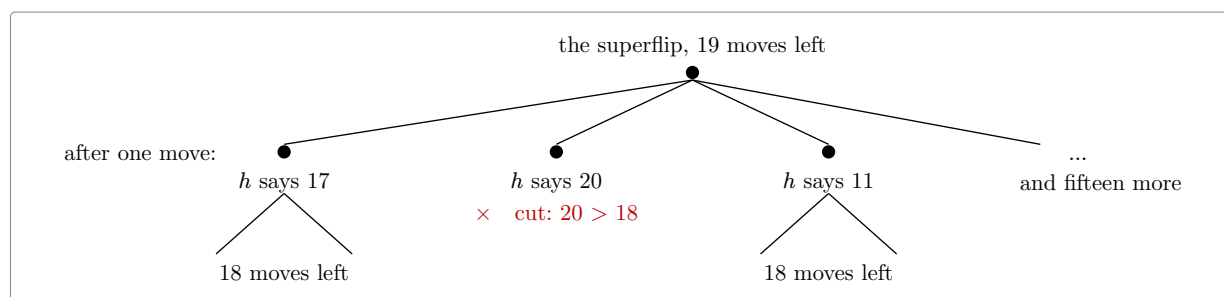


Figure 6: The search, and its scissors.

Such a lower bound is called an **admissible** estimate, and a depth-first search that deepens step by step and prunes can be found for example in Korf's IDA* [3].

3.2 Getting a cheap estimate

The idea behind our estimate is to forget most of the cubes. Keep only part of the information, say how the corners are twisted and where the four middle-layer edges sit, and call what is left a *summary*. Many positions can share the same summary. They will get the same estimate. Moves act on summaries as well as on cubes. Summaries are few, so we can compute the exact distance of each one to the solved summary and keep them all in a table. This gives h : take a position, compute its

summary, and look its distance up in the table. This idea of summary comes from Culberson and Schaeffer, who call such a table a *pattern database* [4], and Korf solved the cube optimally with three of them [5].

We use Kociemba's summary, from his two-phase solver [6], and call it the *phase 1 summary*. Figure 7 shows the three things it records. A corner has one sticker belonging to the up or down face, and that sticker sits in one of three places, which is 0, 1 or 2. An edge is either the right way round or turned over, which is 0 or 1. The four edges of the middle layer occupy four of the twelve edge slots. The figure shades them on the two visible faces, where three of the four can be seen. Nothing else is recorded.

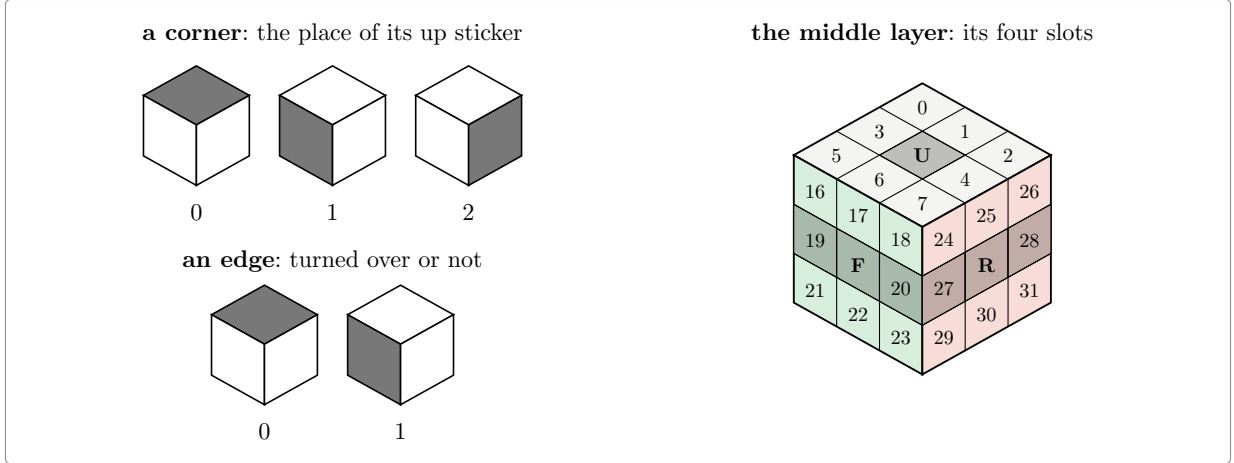


Figure 7: The three things a summary records.

The summary is the product of the three values:

summary	values	
how the eight corners are twisted	2 187	$= 3^7$
how the twelve edges are flipped	2 048	$= 2^{11}$
where the four middle-layer edges sit	495	4 places among 12
the summary	2 217 093 120	

Every summary represents exactly 19 508 428 800 positions. The table records, for each summary, its distance from the solved summary. A distance never exceeds 12, so four bits hold one entry, fifteen of them to a 63-bit machine word: the whole table is *1.18 GB*. The cut is quite effective. A search at depth 14 visits 470 786 nodes only. Without the cut the same tree holds $1.07 \cdot 10^{15}$ of them. We call this summary the *phase 1 summary*, after the first phase of Kociemba's solver [6], and its table of distances the *phase 1 table*.

4 The search in Rocq

4.1 The generic search

The search is generic, **Search.v**: a hundred lines that never mention the cube. It is given a group, a set of moves and an estimate h , and it asks only two things of h :

Hypothesis h1 : $h\ 1 = 0$.

Hypothesis hstep : forall g m, m \in S -> h g <= (h (g * m)).+1.

Fixpoint search (d : nat) (g : gT) : bool := (h g <= d) && ((g == 1) || (if d is d'.+1 then has (fun m => search d' (g * m)) Sseq else false)).

Corollary searchN d g : search d g = false -> g \notin ball S d.

Line by line:

- `gT` is the group the file works in. It is a variable, so nothing here is about the cube; the cube is supplied at instantiation.
- `1` is the unit of that group, the solved position for the cube. So `h 1 = 0` says the estimate of a solved cube is zero, and `g == 1` asks whether the search has arrived.
- `Sseq` is the list of the eighteen moves, in the order the search walks over them, and `S` is the same thing seen as a set.
- `g * m` is the position `g` followed by the move `m`, in the left to right order met above.
- `h g <= d` is the cut. `(h (g * m)).+1` is the estimate after a move plus one, so `hstep` says that one move changes the estimate by at most one.
- `has (fun m => search d' (g * m)) Sseq` tries every move with one fewer move left, and answers as soon as one of them succeeds.

The last line is what we need: if the search returns false, the position is not in the ball of radius d .

4.2 The summary and its table

The estimate is built in a second generic file, `Coord.v`. It is given the summary of a position, the way a move acts on a summary, and the table of distances, with one condition on the summary and two on the table.

```
Variable coord : {perm facelet} -> X.
Variable act    : X -> {perm facelet} -> X.
Hypothesis coordM : forall g m, coord (g * m) = act (coord g) m.
```

```
Variable D : X -> nat.
Hypothesis D0 : D (coord 1) = 0.
Hypothesis Dstep : forall x m, m \in Sset -> D x <= (D (act x m)).+1.
```

`X` is the type of summaries and `coord` gives the summary of a position. `act` plays a move on a summary directly, without going back to the position it came from. `coordM` says the two agree: playing a move and then summarising gives what `act` gives.

The search of `Search.v` is refined to carry the summary beside the position, updating it with `act` at each move. The position is still carried, in order to tell whether the cube is solved. The refined search is given below with simplified names and some details left out (the real one is `searchz3` in `Farpl.v`):

```
Fixpoint search (d : nat) (g : gT) (x : summary) (p : move) : bool :=
  (D x <= d) &&
  ((g == 1) ||
   (if d is d'.+1
    then has (fun m => search d' (g * m) (act x m) m) (allowed p)
    else false)).
```

Here are some explanations:

- `D x <= d` is the cut. It reads the table at the summary `x`.
- `g == 1` asks whether the cube is solved.
- `g * m` moves the position and `act x m` moves the summary. The summary never has to be recomputed from the position.
- `p` is the move just played, and `allowed p` is the list of moves the rules permit after it. This is explained in the next section.

4.3 The table and its two conditions

For the phase 1 summary, `D` is a lookup in the phase 1 table, so `D0` and `Dstep` become two statements about that table:

- the entry of the solved summary is zero;

- every entry is at most one more than the entry reached from it by any of the eighteen moves.

The table is generated by an OCaml program, and it gives the exact distance. The table is not trusted. The two statements above are formally checked by computation. Note that a table full of zeros would also pass our checks but the associated search would prune nothing. The entries of the table do not depend on each other. So the second check, the most expensive one, is cut into slices, one file each. The slices are checked in parallel thanks to the Rocq separate compilation. It takes 10 minutes. Every timing in this note has been measured on the same machine, the *reference machine*: a dual-socket Intel Xeon E5-2667 at 2.9 GHz, twelve cores, twenty-four threads, 62 GB of memory.

5 Optimising

What follows makes the tree search even smaller or the run cheaper.

5.1 Removing redundant moves

Many words lead to the same position, and the search does not have to try them all. Two ideas say which ones may be left out. The first is repetition. After a **U** we do not try **U**, **U2** or **U'**. **U U** is **U2**, so the word is shorter, and shorter words are covered at a smaller depth. That leaves fifteen moves instead of eighteen. Opposite faces give a weaker version of the same argument. **U D** and **D U** give the same position, so we keep only one of the two orders. We play the top, right or front face first. This is what we call the **order convention**. From the third move on it leaves **twelve** moves after a turn of the top, right or front face, and **fifteen** after a turn of the bottom, left or back one. The second idea is symmetry. The superflip is unchanged by all 48 relabellings of the cube. So we need to explore only the turns of one face for the first move. We choose arbitrarily the top one, and again by symmetry we only have to consider **U** and **U2**, since **U'** is the symmetric of **U**. The two ideas collide at the second move. After **U**, repetition removes **U**, **U2** and **U'**, which leaves fifteen. The order convention would remove **D**, **D2** and **D'** as well and leave twelve, but it may not be used here. The first move is already fixed to the top face, and turning the cube upside down takes **D U** back to **U D**. So the bottom face stays, and the fifteen second moves after **U** include **U D**, **U D2** and **U D'**.

The search is then parallelised at depth two. Two first moves times fifteen second moves is thirty *prefixes*. Each is searched on its own to depth 17. They are packed one file per second move, with both first moves inside. The eighteen moves are numbered 0 to 17, three to a face, in the order **U**, **R**, **F**, **D**, **L**, **B**, and a file is named after its second move. Numbers 0, 1 and 2 turn the top face again and cannot be a second move, so the files start at **Runp1_03.v**, whose second move is **R**, and run to **Runp1_17.v**, whose second move is **B'**. Two of them, **D** and **D'**, were measured far the longest of the fifteen, so each is split in two, one file per first move: **Runp1_09a.v** and **Runp1_09b.v**, **Runp1_11a.v** and **Runp1_11b.v**. That balances the load and makes *seventeen files* in all. Our own OCaml program dropped the bottom-face moves, so it searched 24 prefixes where it had to search 30. It ran for hours and gave the answer we expected. The error came out only when the cut had to be proved in Rocq, and the proof could not be written. A cut that is too greedy does not make a search fail. It makes it faster, and it makes it agree with you.

5.2 The effective representation

To run the search its objects have to be in a form Rocq computes with: a position first of all, then a move, the summary of a position, and the distances. We say *array* for the small ones, a position or a move, and *table* for the big ones, the distances and the other functions written out.

A position is a permutation of the 48 stickers, and we write it as its 48 images: the number at place *i* is where the sticker at *i* goes. **Table.v** holds that as a list of 48 numbers, and **tab_ok** is the test that a list is one: 48 entries, each below 48, none twice. A move is written the same way. The move **U** is

2, 4, 7, 1, 6, 0, 3, 5, 32, 33, 34, ...

read off its cycles: sticker 0 goes to 2, sticker 1 to 4, and sticker 8, on the left face, goes round to 32 on the back. Figure 5 shows the same turn the other way round, each square holding the sticker that arrives there, so the picture is this list reversed.

Playing a move on a position is reading one list through the other. To find where sticker i ends up, take entry i of the first list and use that number as the index into the second. Playing **U** twice: entry 0 is 2, entry 2 is 7, so **U2** sends sticker 0 to 7. **Tsearch.v** runs the search of **Search.v** on lists of this kind.

Next come machine integers, 63 bits wide, and *persistent arrays* of them [7]. **Tabi.v** carries a position as an array of 48 such integers. **ti2t** reads an array back as the list it stands for, and **tabi_ok** is **tab_ok** of that list. Each operation has a lemma saying that the bridge may be crossed either way round.

```
Lemma ti2t_comp a b :
  tabi_ok a -> tabi_ok b ->
  ti2t (comp_tabi a b) = comp_tab (ti2t a) (ti2t b).
```

From there on a position is an array of 48 machine integers and a summary is two of them. The phase 1 table is far too big for one array: a Rocq array holds at most 4 194 303 entries, so the table is an array of arrays, cut into chunks of two million words, with fifteen four-bit entries to a word.

The superflip itself goes down that chain. As a permutation it is a product of twelve two-cycles, one for each flipped edge, (133), (39), (425) and so on. **Moves.v** turns those cycles into the list **sftab**, and the list into the array **sfti** the search starts from. Each step has its lemma:

```
Lemma sftabE : superflip = pt 47 sftab.
Lemma sftiE : superflip = pt 47 (ti2t 47 sfti).
```

pt 47 is the permutation a list stands for, so both say that what runs is still the superflip. Its summary is read off the same list, the corner twist by **ctwistt** and the flip-and-slice value by **coordt**. The estimate at the root of the search is then one expression:

```
Dpli (ctwistt sftab) (coordt sftab)
```

The summary is (0,15 732 735). The superflip leaves the corners alone, so the twist is zero, and the second number carries the twelve flipped edges and the four slice slots. The lookup goes through the fold to a four-bit field of one 63-bit integer, and the value there is **ten**. At the root the search therefore knows that at least ten moves are needed, and it has nineteen to spend, so nothing is cut there.

Functions are treated the same way. A function on a finite domain is computed once and written into a table. The action of a move on a summary, the rank of a summary and the symmetry used by the fold are all tables, not computations. The search reads them where the mathematics applies a function. Each of these tables is checked in Rocq, like the table of distances.

The search itself goes the same way. **Fast.v** holds it twice. **searchz3** is the abstract one, written on the objects of the last section. **searchz3n** is what runs, on machine integers and persistent arrays. **FastP.v** ties the two:

```
Lemma searchz3nE T d a p :
  (d <= 63)%N -> fsmoveC -> (p < 7)%N ->
  tabi_ok 47 a -> cubti a -> twP3 a ->
  searchz3n T d (of_nat d) a [::] (init3 a) p
  = searchz3 T d a (init3 a) p.
```

The hypotheses say that the depth fits in a machine integer, that the move table passed its check, and that the array is a well-formed position. Between the two there are seven versions. Each is proved equal to the one before, and together they are 11.9 times faster on one piece at depth 14. The middle versions may be written in any way. The answer stays the one the abstract search gives.

5.3 Two uses of symmetry

The first relabels the position. Rotating the whole cube about a corner axis gives the same position seen differently, and its summary is then another entry of the same table. Each of the three views therefore gives a lower bound on the number of moves left, and so does the largest of the three. That costs three lookups at a position instead of one and buys a sharper cut and a smaller tree. Cube solvers do this all the time, Kociemba's included. What is new here is the proof that the three views are legitimate.

The second relabels the table. The summary is built around the up-down axis: the twist records where each corner's up-or-down sticker sits, the slice where the four edges between the top and bottom faces are. Sixteen of the 48 relabellings keep that axis and turn one summary into another, and they sort the 1 013 760 flip-and-slice values into **64 430 families**, a factor of **15.73**. Two values in one family are the same distance from solved, so one entry per family is enough. The estimate is therefore unchanged, and only the table shrinks. In the code the change is one definition. The lookup was

```
Definition Dpli (tw x : int) : int := plget (plidx tw x).
```

and it became

```
Definition Dplri (tw r : int) : int :=
  plget (plfoldi (frep r) (twsym tw (fsym r))).
```

`frep r` is the family's representative, `fsym r` the symmetry that carries the value to it, and `twsym tw (fsym r)` the twist carried through that same symmetry. Three more reads at every lookup, into a table 15.73 times smaller.

Symmetry-reduced tables are standard in cube solvers. What the development adds is a proof that the folded table still passes **D0** and **Dstep**, and that is all it has to prove. Conditions demanding true distances would have needed a proof that the fold preserves them. That is a harder statement, about the sixteen symmetries and about what sharing an entry between two summaries does. The check is run on the folded table as it was on the flat one, and it is the same check. The fold costs the search 1.61 times at depth 16 and pays everywhere else. A search worker drops from 4.15 GB to **0.85 GB**, so all the pieces run at once instead of in two waves. Checking the table drops from about 5.4 processor hours to **1.35**.

6 The development and its cost

The statement proved at the top of the chain is

```
Theorem superflip_plfar_real : superflip \notin ball Sset pldepth.
```

In words, the superflip is not within `pldepth` moves of the solved cube, where one script sets the depth before the run. It has **no hypotheses left**, and nothing in the chain is admitted: asking Rocq what the proof assumes reports only the primitives of its machine-integer and array interface. At depth 19 it says that the superflip cannot be solved in 19 moves, and two lines in **Diam20.v** turn that into **God's number** ≥ 20 , after checking that the searches really were run at 19.

It is forty-six hand-written files, 12 725 lines about the cube and 1 308 more of machine-integer toolbox, beside 156 000 lines of generated tables in the repository and 165 MB of them too big to store. The cube and its symmetries are **Rubik333.v**, **Sym.v** and **Ball.v**, the abstract search **Search.v** and **Coord.v**, the representations **Table.v** and **Tabi.v**, the summary and its table **Coordfs.v** and **Phase1.v**, and the search on the real data **Farpl.v**, **Fast.v**, the seventeen pieces **Runp1_03.v** to **Runp1_17.v** and **Diam20.v**. The checks live in certificate files of their own, each behind its own **Qed**, and **README.md** lists everything with the scripts that run it.

Building the tables costs the same whatever radius is searched afterwards. Measured end to end from a clean tree on the reference machine:

	wall clock	processor time
emitting the tables and compiling them to native code	17 min 21	52 min 13
the coordinate and summary tables	22 min 46	21 min 15
the search and the estimate, over no data at all	11 min 02	30 min 45
the four certificates for the move and distance tables	56 s	2 min 27
the fold: twelve checks and twenty-seven slices	9 min 52	47 min 57
the shape check against the dummy table	18 s	16 s
in total	1 h 02	2 h 35

The second line runs on one core, so more cores do not help. The first and fifth are mostly the OCaml compiler turning a table into native code, and together they take 100 of the 155 processor-minutes.

The search itself visits 146 065 078 152 positions, from 5 575 767 076 in the smallest piece to 10 554 835 820 in the largest, and the tree grows by 12.22 from one level to the next between depths 17 and 19. We measured the run twice, once before the fold and the two reductions and once after. It is the same theorem both times.

radius 19, search depth 17	before	after
pieces	18	17
workers	9	18
memory per worker	4.15 GB	0.85 GB
wall clock	11 h 13	6 h 36
processor time	85 h 11	87 h 36

The fold halves the wall clock and leaves the processor time where it was. What it buys is memory: at 0.85 GB a worker, all seventeen pieces run at once, where 4.15 GB allowed only nine. So the proof costs 87 processor-hours and one night, and that is a measured cost.

7 Counting in quarter turns

We have proved that God's number is at least 20. We now count the moves differently: in quarter turns there are twelve moves, the six faces one way and the same six back, and a half turn is two moves. The answer in that count is **26** (cube20.org), and we prove the lower half: one position cannot be solved in 25 quarter turns.

7.1 The position, and 25 down to 24

The superflip is only 24 quarter turns from solved, so it is not far enough. Reid posted a better position to the Cube-Lovers list in August 1998 [8]: the superflip composed with four spot, the four-spot pattern with the superflip on top of it. In the following, we call it **superflip4**.

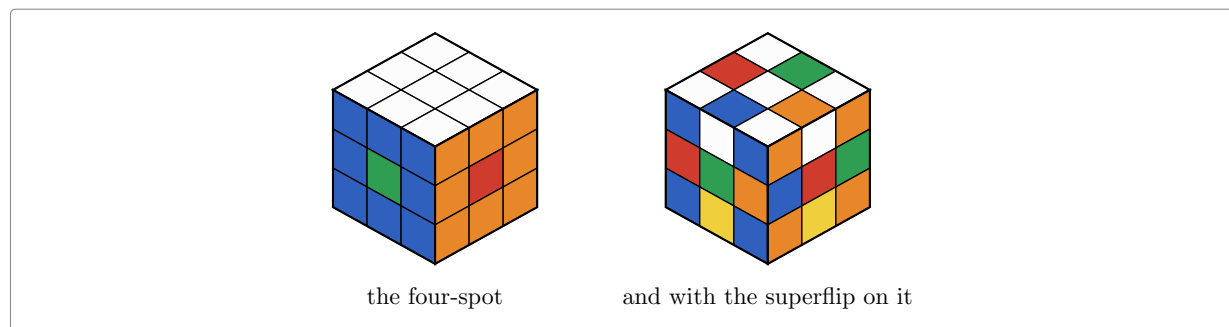


Figure 8: The four-spot, and superflip4.

Superflip4 is 26 quarter turns from solved. Here is the word given for it, with the half turns written out:

U U D D L F F U' D R R B U' D' R L F F R U D' R' L U F' B'

The searches only need to go to depth 24. A quarter turn is five four-cycles of the 48 stickers, so it is an odd permutation and each move changes the parity of the position. Superflip4 is even, like the solved position, so every word for it has even length.

7.2 The six prefixes

Every shortest word for superflip4 can be rewritten, at the same length, so that it begins with one of six prefixes [8]:

$$\begin{array}{ccc} \text{R U} & \text{R' U D} & \text{R' U F'} \\ \text{R' U R'} & \text{R' U B'} & \text{R' U L'} \end{array}$$

The searches start from those six. In order to prove this fact, we split the twelve quarter turns into two sets:

$$\mathcal{A} = \{ \text{U, U', D, D'} \}, \quad \mathcal{C} = \{ \text{R, R', F, F', L, L', B, B'} \}.$$

A word made only of turns from $\mathcal{A}$ leaves the four middle edges untouched, and superflip4 has them flipped, so no such word gives superflip4. A word made only of turns from $\mathcal{C}$ never flips an edge, and superflip4 has every edge flipped, so no such word gives it either. Any word for superflip4 therefore uses turns from both sets, so somewhere in it a two-letter subword has one turn from each set.

Each of the six prefixes starts with a turn from $\mathcal{C}$ followed by a turn from $\mathcal{A}$, so we want a word for superflip4 that starts with such a subword. Recall the word we gave for it:

$$\text{U U D D L F F U' D R R B U' D' R L F F R U D' R' L U F' B'}$$

Reading along it, four two-letter subwords have a turn of $\mathcal{C}$ followed by a turn of $\mathcal{A}$, and five have them the other way round. We take the first of the four, at the seventh and eighth turns, F then U', and call that subword q . A word may have subwords of the second kind only. We would then invert it first: superflip4 is its own inverse, so the inverse is again a word for it, of the same length, and inverting turns every subword of the second kind into one of the first. Here the inverse is

$$\text{B F U' L' R D U' R' F' F' L' R' D U B' R' R' D' U F' F' L' D' D' U' U'}$$

with, as expected, five subwords of the first kind and four of the second.

Let us go back to our two-letter subword q . Six turns stand before it. Write x for them, U U D D L F, and w for what follows q , so that $xqw = P$, where P is superflip4. Then $qw = x^{-1}P$, and therefore

$$qw(P^{-1}xP) = x^{-1}PP^{-1}xP = P.$$

$P^{-1}xP$ is not longer than x :

$$P^{-1}xP = (P^{-1}\text{U}P)(P^{-1}\text{U}P)(P^{-1}\text{D}P)(P^{-1}\text{D}P)(P^{-1}\text{L}P)(P^{-1}\text{F}P) = \text{U U D D R B}.$$

The first step is done: qw followed by those six turns is a word for superflip4 of the same length, beginning with q .

$$\text{F U' D R R B U' D' R L F F R U D' R' L U F' B' U U D D R B}$$

The second step renames the faces. A relabelling of the cube that leaves superflip4 unchanged carries a word for it to another word for it, of the same length. Sixteen of the 48 relabellings do that. The superflip is unchanged by all 48, but the four-spot is not: it leaves the top and bottom faces alone and exchanges the colours of the other four in pairs, so it singles out the up-down axis, the line through the centres of the top and bottom faces. A relabelling that moves that axis carries the four-spot to the same pattern about another axis, which is another position. The sixteen that keep the axis are the ones that leave superflip4 alone. They make the letter of q from $\mathcal{A}$ into U, and the letter from $\mathcal{C}$ into R or R', so q becomes R U or R' U. In our example q is F U'. The mirror that swaps left and right fixes the front face and reverses the direction of every turn, so F U' becomes F' U. A quarter rotation about the up-down axis then carries the front face to the right one, and F' U becomes R' U.

A word starting with **R U** needs nothing more, which is the prefix of two turns. When it starts with **R' U** we look at the third turn. Six of the possible turns give words that a relabelling or an inversion brings back to the **R U** case, and five do not. Those five are the prefixes of three turns.

7.3 The summary, and its table

The estimate is built as before. We take the summary Reid uses in the quarter-turn count. Against the phase 1 summary of the first bound, it keeps the corner twist, the same 2 187 values, and replaces the edge flips and the slice by where the four middle edges sit with their flips, and by which four corner places hold the top corners.

summary	values	
where the four middle-layer edges sit, each of them the right way round or not	190 080	$= 24 \cdot 22 \cdot 20 \cdot 18$
which four corner places hold the four top corners	70	4 places among 8
how the eight corners are twisted	2 187	$= 3^7$
the three together	29 099 347 200	

Each of the four edges can sit in any free slot, either way round. That gives 24 choices for the first, and two fewer for each of the others, since a slot taken is taken whichever way round the edge in it lies. A summary is the coset of a subgroup H of the cube group, which is why the file names of this section start with H . It is not the subgroup of the first bound, and the summaries are thirteen times as many, 29 billion against 2.2 billion.

The table holds the distance from solved of each of the 29 billion summaries. The number of summaries at each distance agrees with the column published in 1998, and we check that first. The table is then folded. The sixteen symmetries that keep the up-down axis sort the 190 080 edge values into 12 094 families, a factor of 15.72, and one entry is kept per family. That is 883 MB, and 3.86 GB once loaded into the prover.

7.4 The theorem, and its cost

The position is settled both ways in `HFinal.v`:

Theorem `targ_dist` : `targ \in ball Sq 26 /\ targ \notin ball Sq 25`.

`targ` is `superflip4` as the searches meet it, and `ball Sq n` is the set of positions within n quarter turns. The word gives the first half and the six searches give the second. The bound follows:

Theorem `qdiam25` : `~ diam_le Sq 25`.

`diam_le Sq 25` says every position is within 25 quarter turns, and the line says it is not. Rocq reports only the primitives of its machine-integer and array interface. The work is eighteen hand-written files and 6 008 lines. The argument above is in `HProp2.v`, the search in `HSearch.v`, the sweeps in `HSweep.v`, and the bound in `HAll.v`.

	wall clock	processor time
building the table, in OCaml	9 min 50	1 h 43
the same table as fifty-nine Rocq files	3 h 13	6 h 50
the three sweeps over the move tables		1 min 20
the sweep over the distance table, twelve jobs	2 h 51	31 h 42
the six searches, seventy-two pieces, twelve workers	4 h 00	45 h 54
the whole chain in Rocq	10 h 32	87 h 29

The last row is measured end to end, from a directory where nothing is built. It is not the sum of the others. The OCaml table of the first row is built once by hand, and it is not part of that run.

The sweeps and the searches are nearly nine tenths of the cost, and checking the table rather than trusting it costs about two thirds of what the searches cost.

8 One coset of the upper half

So far we have dealt with lower bounds. This last section is an attempt at the real computation, the upper bound.

Here is what it proves, in full. Take the superflip. Play on it any sequence of the ten moves **U**, **U2**, **U'**, **D**, **D2**, **D'**, **R2**, **L2**, **F2** and **B2**. There are 19 508 428 800 positions that can be reached that way. Every one of them is solved in twenty moves or fewer.

The computation that proves it is not the search of the earlier sections. There the search was looking for a manoeuvre and had to come back empty; the whole difficulty was that it must have left nothing out. Here it is the opposite. Each of the 19 508 428 800 positions gets one bit, the computation sets the bit of every position it can solve in twenty moves, and the theorem follows when no bit is left clear. What runs for six hours is a marking. The rest of this section is how the bits get set.

8.1 Cosets

Ten of the eighteen moves generate the subgroup H of Kociemba and Thistlethwaite: **U**, **U2**, **U'**, **D**, **D2**, **D'**, **R2**, **L2**, **F2** and **B2**. They are the moves that do not change the phase 1 summary: they do not twist a corner, they do not flip an edge, and they keep the four middle edges in the middle layer. A coset of H is the set of positions we get from one position using only these ten moves. A coset is therefore one value of the summary, and there are as many cosets as summaries. Every position of the cube lies in exactly one coset.

	count
positions of the cube	43 252 003 274 489 856 000
cosets	2 217 093 120
positions in a coset	19 508 428 800
cosets we did	1

The published proofs do not solve all 43 quintillion positions one at a time. They solve a whole coset at once. One search settles every one of the 19 508 428 800 positions in it. The positions in a coset are not all the same distance from solved. The search shows that none of them is more than 20. A symmetry of the cube carries one coset to another, and the image is solved by the same manoeuvres relabelled, so only one coset per symmetry class is searched. Two things are then needed. The cosets searched must cover every class, and each search must settle its whole coset.

Canon.v proves the first, and it is not a computation. As the representative of a class we take its least member, in the order the type of positions already carries. Every class is covered, because a finite set that is not empty has a least member. The file is eighty lines and assumes nothing.

The second is the computation itself. Rokicki, Kociemba, Davidson and Dethridge ran it over 55 882 296 families of cosets. It took about a billion seconds of processor time, more than thirty processor years, given by Google. We cannot repeat that. What we can do is one coset, to see what one costs and whether the pieces are in place. We did the superflip's, because we already had the superflip from the lower bound.

8.2 The coset as one map

A position of the coset is named by three numbers: how the eight corners of the top and bottom layers sit, how the eight edges of those layers sit, and how the four middle edges sit. That is 40 320 by 40 320 by 24 arrangements, but half of those triples cannot occur: on the cube the corners and the edges are always permuted with the same sign. The map holds one bit for each of the 19 508 428 800 triples that are left.

The bits are laid out the way Rokicki's own program lays them out, and the layout is what makes the next subsection cheap. A page is one arrangement of the corners. Inside a page, a group is a pair of arrangements of the outer edges, the two that differ by exchanging one pair of cubies, and the group's twenty-four bits are the twenty-four arrangements of the middle edges, the twelve even ones low and the twelve odd ones high. The parity says which arrangement of the pair a bit stands for, which is how the impossible half disappears with nothing left to store.

A machine word holds forty-eight bits, so one word holds the same group on two pages, the corner arrangements of even and odd rank, the odd one in the top half. The map is then 20 160 times 20 160 such words: 406 425 600 words, 3.25 GB. An array in Rocq may hold 4 194 303 entries, so the map is an array of 194 chunks of two million words. The map and its indexing are `Row.v` and `RowMap.v`.

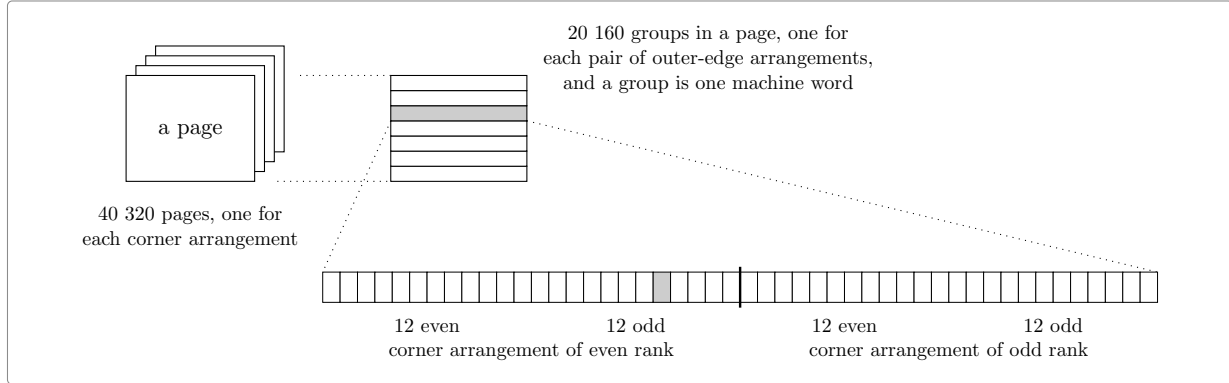


Figure 9: The map, from the outside in. A page for each arrangement of the corners, a group in the page for each pair of outer-edge arrangements, and a bit in the group for each arrangement of the four middle edges, the twelve even ones low and the twelve odd ones high. One word holds the same group on the two pages of a pair.

The invariant the whole proof turns on is one line: a map is **sound at d** when every bit it has set is a member of the coset that is within d moves of solved. The map the run starts from has one bit set, the superflip's own, and is sound at nought. Each of the twenty steps below takes a map sound at d to a map sound at $d + 1$. If the map that comes out has every bit set, then every member is within twenty moves, and that is the theorem.

8.3 A level

The run is twenty levels, level d turning a map sound at $d - 1$ into a map sound at d . A level does two things, and they divide the manoeuvres of length d between them.

- The **prepass** plays each of the ten moves of H on the whole map at once. It accounts for every manoeuvre whose last move is in H , and that is nearly all of them.
- The **search** then looks only for the manoeuvres of length d whose last move is not in H . It walks them from the superflip one position at a time, the way the searches of the earlier sections do.

In `RowSrch.v` the run is seven lines. It carries the map, the map it reads while it writes, and the number of bits the last level left, which is what decides whether the cuts below are on.

```
Fixpoint runsk (n : nat) (d : nat) (n0 : int) (m dst : rmap) : rmap :=
  if n is n1.+1 then
    let m' := levelsk (Uint63.ltb ncutb n0) d.+1 m dst in
    runsk n1 d.+1 (mcount m') m' m
  else m.
```

`levelsk` is the level: the prepass first, then the search if this level is still being searched. Counting the bits, `mcount`, is a sweep of the whole map, and the run pays for it once a level.

8.4 The prepass

The prepass is what makes a whole coset affordable, and it is the one part of the computation that has no counterpart in the lower bounds.

Playing a move of H on a position of the coset does three separate things to the three numbers that name it. The corner arrangement goes to another corner arrangement, so a page goes to a page. The outer-edge pair goes to another pair, so a group goes to a group. The middle arrangement goes to another middle arrangement, so the twenty-four bits of the group are rearranged among themselves, and that rearrangement depends only on the move and the group it came from. It is a table lookup and a shuffle of one machine word.

So the prepass never takes a position apart. It never builds a cube, never ranks one, never looks a position up. For each of the ten moves it reads the whole map and writes the whole map, and that is all of it. Ten passes over 3.25 GB buy every manoeuvre of this length that ends in a move of H , and there are billions of those.

`RowMap.v` has the prepass and the proof that it keeps the map sound. `RowLvl.v` has it again, written so that a page's chunk is fetched once and put back once instead of once a word, and proves the two are the same function, so nothing about the cube is proved twice.

8.5 The search

The search is a depth-first walk from the superflip, like the ones in sections 3 to 5, with two differences. It carries the position it has reached, and when it has used up its depth it asks whether that position is a member of the coset. If it is, the position's three numbers are computed, and the bit they name is set. That is the leaf, and it is two lines of `RowSrch.v`:

```
else if csolved c
  then let: (pg, gr, bt) := plc (tomemb x) in mmark m pg gr bt
  else m
```

The membership test is made on the position itself, which the search carries anyway, and it costs one comparison. It is worth saying why it is not made on the phase 1 table, which is right there and would give the same answer for a handful of nanoseconds less. The table is an estimate. A table of zeros is a legal estimate – it passes both conditions of section 4.3, so nothing rules it out – and with a table of zeros every position would look like a member, every bit would be set at once, and the theorem would say nothing. Reading membership off the table would make the theorem depend on the table being sharp, which nothing proves. Reading it off the position makes it depend on nothing.

The other use of the table is a cut, and that one is sound. At each node the search looks the position up and gets a number that is never larger than the number of moves needed to bring it into H . If that number is larger than the moves left, the branch cannot reach the coset and is dropped. An estimate that is too small only cuts less than it could.

8.6 The cuts and the early stop

A search that offered all eighteen moves at every node would never finish. Four things cut it down, three of them Rokicki's.

The first is the moves worth trying. The phase 1 table of section 5.3 is already folded by the sixteen symmetries; the copy the search reads, `RowMask.v`, carries beside each distance the set of moves that bring the position nearer H and the set that at least do not take it further. Which set is wanted depends on how many moves the search has to spare. A node then offers three or four moves instead of eighteen. The entry is twenty-eight bits, two to a machine word: four bits of distance and four for each of the six faces. Written out one bit a move it would take forty-one. A half turn is a quarter turn twice, so the two moves of one face cannot differ by more than one step, which leaves fifteen of the twenty-seven ways a face can go, and fifteen fit in four bits.

The second is that the last move of a manoeuvre is never a move of H . A manoeuvre that ends in H is a shorter one followed by moves of H , and the prepass has already played those.

The third is that low down, a move must go straight at H . Once the moves left and the distance to H add up to less than five, only the moves that shorten the distance by one are followed. A move that wastes a step down there ends in moves of H , so the prepass catches that manoeuvre too.

The fourth is where the search stops altogether. It runs on the first sixteen levels and not on the four above them, and on level sixteen it does not even run out: it stops as soon as the map holds 167 million bits plus a third of what that level's prepass left. Those are Rokicki's numbers, and every published time of his was measured with them on. What is left is finished by the prepasses of levels seventeen to twenty.

The second and third cuts only come on once the map holds more than six million bits, which on this coset is not before level fourteen, and that too is his rule.

Every one of these four is safe, and safe in a way none of the cuts in the earlier sections were. There, a cut that lost a manoeuvre would have lost the proof, and each one had to be shown to lose nothing. Here nothing is ever proved about what the search covers. What is proved is that the map filled. A cut that throws a manoeuvre away can only make the map fill later, or leave a bit clear at the end – it can never set a bit that should not be set. This is why a coset is easier to prove than a lower bound, and it is why the four cuts above needed no argument at all. The one thing that did need an argument is marking, which is the subsection above.

8.7 The members left over

Stopping the search at level sixteen, and stopping level sixteen early, leaves members the run never reaches. When the two runs ended, thirty-two bits of the 406 425 600 words were still clear. They are simply the members that no manoeuvre the cut search kept, followed by moves of H , happened to reach. Searching deeper would leave fewer and cost hours; each one left over costs a line.

So we give each of the thirty-two a manoeuvre of twenty moves by hand, in `RowWits.v`. Twenty-eight of them came from our prototype and the last four were found one at a time. None of that is believed. `RowWitsChk.v` plays each of the thirty-two manoeuvres back on the member it claims to solve and asks for the solved cube. If the producer had made a mistake, or given a manoeuvre of twenty-one moves, or named the wrong member, the check would fail; it cannot make the theorem false. The replay is also the only thing in the development that tests the two ideas of which facelet is which – the prototype's, which found the manoeuvres, and Rocq's, which builds the member – against each other.

The thirty-two are marked into the map at the very end, by `RowMark.v`, and not at the start. A map sound at d claims every bit it has set is within d ; a witness is within twenty, so seeding one before the run would make level one claim its neighbours are within one, and they are not. Marking at the end is sound because the map is already sound at twenty when the marks go in.

8.8 The run, and its one boolean

Everything above is one boolean. `RowFoldCubDefI.v` builds the map, runs the twenty levels, marks the thirty-two and asks whether every bit is set:

```
Definition rowfulli : bool := mfullf ffuli ycwitsoi.
```

```
Lemma rowfulliE : rowfulli = true.
```

```
Proof. Time native_cast_no_check (erefl true). Qed.
```

That `Qed` is the six hours. The file that holds it, `RowFoldCubBoolI.v`, is one `Require` and one `Lemma`: a file that runs loads no proof, and a file that proves runs nothing. Everything else – that the prepass

keeps the map sound, that the search marks only members, that a full map is the theorem – is proved beside it, of a map and a run the proof never evaluates.

8.9 Folding the map

The map can be made smaller, and we made it smaller to see what its size does to the run. Both maps were searched, the folded one and the unfolded one, and the two runs are compared at the end of the section.

The map is 40 320 pages, one for each arrangement of the corners.

The fold works because of the position we chose. Sixteen of the 48 renamings of the cube keep the top and bottom faces in place. Each of them sends the ten moves to the ten moves, so it maps H to itself, and each leaves the superflip unchanged, so it maps the superflip’s coset to itself. A member and its image under such a renaming need the same number of moves, so two pages related by a renaming hold the same answer. One page of each family is then enough: 2 768 of the 40 320, a factor of 14.6. A level of the run is one pass over the map, so there is 14.6 times less of it to walk. The price is undoing a renaming whenever a kept page is read.

The kept pages go in pairs of their own, and a pair shares one word as it does on the unfolded side: 1 496 words in place of 2 768. Two hundred and twenty-four of the pages are their own partner and use half a word.

The fold has to be proved as well as written. A renaming sends a member of the coset to a member of the coset. Undoing it gives back the position the page stood for. A map sound after one level is sound after the next. That is the largest single part of the coset’s proof.

8.10 The reused and the new

We reuse nearly everything. The abstract search and its contract come over unchanged, and so do the cube, the permutations, the machine-integer tools, the phase one table and its generator. Each of the new tables has a check of its own, in a file of its own, so that a failure names the table that failed.

Five things are new, and only one of them is a search.

- The rank and the sign of a permutation. Rocq’s library has both, but for permutations that do not compute, so we need their effective version on machine integers.
- The link between a position and its three ranks. The map’s bits are triples of numbers, so what the run proves is a statement about triples. To read it as a statement about the cube we have to turn a triple back into the position it stands for. The other way we already had.
- The pass that steps a whole map one move at a time.
- The fold by the sixteen renamings.
- Manoeuvres given by hand for the members the run does not reach.

8.11 The files and the cost

The coset adds sixty-nine hand-written files and 17 386 lines, besides the generated tables. The coset and its members are `Row.v`, `RowMemb.v` and `RowInst.v` the ranking and the moves `Lehmer.v` with the `RowPart`, `RowMove` and `RowTab` groups; the map and the search `RowMap.v`, `RowRun.v`, `RowSrch.v` and `RowFinal.v` the fold the `RowFold` group; and the two runs the `RowCub` and `RowFoldCub` groups, ending in `RowCubDoneI.v` and `RowFoldCubDoneI.v`. `README.md` lists every one of them with what it does.

The search ran twice, over the folded map and over the unfolded one, with the same search in both, and both times it filled the map.

the run	wall clock	processor time	peak memory
over the folded map	5 h 53	5 h 51	–
over the unfolded map	7 h 52	7 h 48	23.3 GB

The fold is worth 1.3 times on the wall clock and 1.3 times on processor time. The run does not follow the size of the map, which is 13 times smaller. Most of the work is the search at the deepest levels, and that is the same tree on both sides. The map sets the memory. The unfolded map is 3.25 GB against 248 MB, and a level reads one map while it writes the other, so the unfolded run needed 23.3 GB of a 62 GB machine. With the garbage collector set to work harder, the same run peaks at 13.9 GB and takes four per cent longer.

Two earlier folded runs say where the time went. Over words of twenty-four bits, holding one corner arrangement each, the run took 6 h 00. The same run with the depth left as a unary numeral instead of a machine integer took 8 h 13, so the unary depth cost 2 h 13.

The phase one table is 2.9 GB of Rocq source and 4.5 GB once checked, 8 h 48 of processor time. It is generated once and shared with the lower-bound work.

The statement has no hypothesis left.

Theorem `real_row_superflip_fold_runi m :`
`m \in H -> superflip * m \in ball Sset 20.`

H is the subgroup above, so the line says that every position of the superflip's coset is within twenty moves.

9 Conclusion

We prove two lower bounds. No position of the cube is solved in 19 face turns, and none in 25 quarter turns. In each case one position is the witness, the superflip for the first and the four-spot with the superflip on it for the second. In each case the proof is a search that comes back empty over a table of estimated distances.

We do not prove that 20 and 26 always suffice. The upper half is a different computation. It is not one search that finds nothing, but two billion searches that must each find everything. We did one of the two billion, and the section above says what it cost.

The three share a trunk. The cube itself, as permutations of the forty-eight facelets, with the eighteen moves and the group they generate. Balls, and what it means for a position to be within d moves. The abstract search, whose contract is that a false answer is a proof. The rule that any summary of a position, together with any table that passes one check, gives an estimate that is never too big. And the tables themselves, held as machine integers in arrays rather than as lists of unary numbers.

What each of the three needed of its own:

- **The twenty face turns.** The phase one summary, which is the edge flips and the slice, its table, and the certificate that checks the table. Three viewing angles of the same search, and seventeen pieces run side by side.
- **The twenty-six quarter turns.** The cut to six prefixes, which is the one piece of the development argued by hand rather than computed, and the parity argument that turns 25 into 24. A second and much larger summary, 29 billion values, with a table of its own and a sweep of its own.
- **One coset.** A coset held as a map of bits rather than as a tree of positions. That needs five new pieces. The rank and the sign of a permutation on machine integers. The link between a position and its three ranks. A pass that steps a whole map one move at a time. The fold by the sixteen renamings. And manoeuvres given by hand for the members the run does not reach.

The same search written in OCaml is about three times faster than the one Rocq runs. We ran both at radius 19 on the reference machine. The OCaml program visits 146 065 078 152 positions in 26.4 processor-hours, which is 0.65 microseconds a position. Rocq takes 87.6 processor-hours over the same tree, which is 2.16. A factor of **3.3**.

We do not assume that the two walk the same tree. We divide each of the seventeen Rocq pieces by the positions its OCaml counterpart visited. The result is between 1.98 and 2.52 microseconds, over pieces that differ in size by a factor of two. A Rocq search that cut differently anywhere would show as scatter there, and there is none. So the run takes a night because the tree holds 146 billion nodes, not because the prover is slow: in OCaml the same tree still costs 26 processor-hours.

The whole development, counted in hand-written Rocq and leaving out the generated tables, is 37 898 lines. Each line of the table counts what that piece adds to the ones above it.

	files	lines
the superflip, for the twenty face turns	53	14 504
the four-spot, for the twenty-six quarter turns	18	6 008
one coset of the upper bound	69	17 386
in all	140	37 898

This development was written with the help of Claude, Anthropic’s coding assistant.

The sources are at github.com/thery/DoubleCover/code/Rubik, with the note, its figures and Reid’s transcribed post beside them.

References

1. The Rocq Development Team: The Rocq Prover, <https://rocq-prover.org/>
2. Mahboubi, A., Tassi, E.: Mathematical Components. Zenodo, 2021, <https://doi.org/10.5281/zenodo.4457887>
3. Korf, R.E.: Depth-first Iterative-deepening: An Optimal Admissible Tree Search. *Artificial Intelligence*. 27, 97–109 (1985). [https://doi.org/10.1016/0004-3702\(85\)90084-0](https://doi.org/10.1016/0004-3702(85)90084-0)
4. Culberson, J.C., Schaeffer, J.: Pattern Databases. *Computational Intelligence*. 14, 318–334 (1998). <https://doi.org/10.1111/0824-7935.00065>
5. Korf, R.E.: Finding Optimal Solutions to Rubik's Cube Using Pattern Databases. In: *Proceedings of the Fourteenth National Conference on Artificial Intelligence (AAAI-97)*. pp. 700–705 (1997)
6. Kociemba, H.: The Two-Phase Algorithm, <https://kociemba.org/cube.htm>
7. Armand, M., Grégoire, B., Spiwack, A., Théry, L.: Extending Coq with Imperative Features and its Application to SAT Verification. In: *Interactive Theorem Proving (ITP 2010)*. pp. 83–98. Springer (2010)
8. Reid, M.: superflip composed with four spot. Post to the Cube-Lovers mailing list, 2 August 1998, The address cited by Rokicki et al. no longer resolves; this is a transcription, <https://github.com/thery/DoubleCover/blob/main/doc/reid-1998-fourspot.md>